

# Capitolo 06 — Il ciclo tra le due memorie

|                    |                                                                        |
|--------------------|------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book            |
| <b>Versione</b>    | FROZEN-06                                                              |
| <b>Data master</b> | 2026-08-28                                                             |
| <b>Stato</b>       | FROZEN                                                                 |
| <b>Source path</b> | wcm/documentation/process-memory-book/chapters/06_ciclo_dual_memory.md |
| <b>Source SHA</b>  | 2b56e3e                                                                |
| <b>Release</b>     | 2026-08-29T07:40:00.546Z                                               |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

**Stato:** FROZEN **Blocco:** 1 — Fondamenti + Dual Memory **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

## 6.0 La memoria non è un luogo: è un movimento

Nei tre capitoli precedenti abbiamo separato i due lati della Dual Memory.

Da una parte c'è la **Working Memory**: il contesto vivo nel quale WCM comprende, ragiona, interpreta intenzioni e mantiene le sfumature necessarie al lavoro corrente.

Dall'altra c'è la **Persistent Organizational Memory**: la memoria strutturata che conserva ciò che deve sopravvivere, essere ritrovato, verificato e governato nel tempo.

Se ci fermassimo qui, avremmo però soltanto due contenitori.

Il WCM nasce invece dall'interazione continua tra i due.

La vera unità architeturale non è quindi:

WORKING MEMORY  
+  
PERSISTENT MEMORY

ma:

WORKING MEMORY  
↓  
CONSOLIDATION  
↓  
PERSISTENT ORGANIZATIONAL MEMORY  
↓  
SELECTIVE RETRIEVAL  
↓  
WORKING MEMORY

La continuità non deriva dal fatto che il sistema ricordi tutto.

Deriva dal fatto che sa **trasformare il presente in memoria durevole e riportare nel presente soltanto la memoria necessaria**.

È questo movimento bidirezionale che rende la Dual Memory una vera architettura operativa.

## FIG-001B — Il ciclo completo della Dual Memory

FIG-001B — Dual Memory Architecture, versione operativa

La figura mostra due flussi distinti.

Il primo va dalla Working Memory alla memoria persistente:

```
Working
→ Delta Detection
→ Classification
→ Consolidation
→ Persistent
```

Il secondo torna dalla memoria persistente alla Working Memory:

```
Persistent
→ Index / Navigation
→ Progressive Retrieval
→ Context Sufficiency
→ Working
```

I due flussi non sono simmetrici.

Nel primo il problema è decidere **che cosa merita di sopravvivere**.

Nel secondo il problema è decidere **che cosa serve adesso**.

---

### 6.1 Working → Persistent: come il presente diventa memoria

Ogni sessione di lavoro produce continuamente informazione.

Ma solo una parte di quell'informazione modifica davvero ciò che l'organizzazione deve ricordare.

Durante una conversazione possono comparire:

- domande;
- ipotesi;
- correzioni;
- alternative;
- nuove informazioni;
- decisioni;
- cambi di stato;
- requisiti;
- evidenze;
- nuovi vincoli;
- learning;
- output approvati.

Il primo compito del ciclo è distinguere il **movimento cognitivo temporaneo** dal **delta organizzativo durevole**.

Questa distinzione è ciò che impedisce due errori opposti:

SALVARE TUTTO

e:

NON SALVARE ABBASTANZA

Il flusso corretto parte da una domanda semplice:

**Che cosa è cambiato, rispetto alla memoria persistente rilevante, che una futura ripresa del lavoro deve poter conoscere?**

Se la risposta è “nulla di materiale”, non serve creare persistenza per rituale.

Se la risposta è “qualcosa di materiale è cambiato”, inizia il consolidamento.

## 6.2 Delta Detection: trovare ciò che è davvero cambiato

La **Delta Detection** è l'atto di identificare la differenza significativa tra il contesto corrente e la memoria persistente pertinente.

Il delta non coincide con il messaggio più recente.

Un singolo messaggio può contenere:

- una conferma;
- una correzione;
- una nuova decisione;
- un dettaglio irrilevante;
- una proposta che non modifica nulla;
- più cambiamenti di natura diversa.

Per questo WCM non tratta il testo come unità organizzativa.

Tratta come unità il **cambiamento di significato o di stato**.

Esempio astratto:

MEMORIA PERSISTENTE  
Limite corrente = 100  
  
WORKING MEMORY  
"Facciamo 120, da ora."

Se la frase proviene dall'autorità appropriata e produce una decisione valida, il delta non è:

nuova frase

ma:

decisione corrente:  
100 → 120

Un altro esempio:

```
MEMORIA PERSISTENTE
Stato workflow = ACTIVE
Next transition = REVIEW

WORKING MEMORY
Review completata con esito PASS
```

Il delta non è “abbiamo fatto una review”.

È:

```
last_completed_transition = REVIEW
next_transition = ...
```

La Delta Detection traduce quindi il linguaggio vivo del presente in una domanda organizzativa:

**| Quale proprietà persistente deve ora essere diversa?**

---

## 6.3 Classification: capire che tipo di delta abbiamo davanti

Una volta individuato un cambiamento, WCM deve classificarlo.

Perché la stessa frase può avere effetti molto diversi a seconda della natura del contenuto.

**PROC-006 Memory Consolidation & Consistency Loop** prevede almeno queste categorie:

- temporaneo / ragionamento;
- ipotesi / proposta;
- decisione;
- stato / fatto;
- execution state / workflow checkpoint;
- requisito / vincolo / rischio;
- evidence;
- learning / metodo;
- elemento superseded.

La classificazione risponde alla domanda:

**| Che cos'è questo cambiamento?**

Non ancora:

**| “Dove lo salviamo?”**

Prima dobbiamo sapere che tipo di cosa è.

Questa separazione è essenziale.

Se una proposta viene classificata come decisione, nasce falsa authority.

Se un risultato viene classificato come baseline, l'evidence modifica il metodo senza governance.

Se un cambio di workflow viene salvato come semplice nota, la sessione successiva potrebbe non sapere dove riprendere.

La classificazione è quindi il primo confine tra cognizione fluida e organizzazione governata.

---

## 6.4 Authority / Status Check: il significato non basta

Dopo la classificazione viene una verifica ulteriore.

Un contenuto può sembrare una decisione, ma non avere l'authority necessaria.

Può sembrare corrente, ma essere una proposta.

Può essere corretto, ma non ancora approvato.

WCM deve quindi verificare almeno:

- chi ha prodotto il contenuto;
- quale ruolo possiede;
- quale authority è richiesta;
- quale status ha il contenuto;
- quale fonte corrente verrebbe modificata;
- se esiste un gate applicabile.

Esempio:

"Secondo me dovremmo cambiare X"

può essere:

PROPOSTA

mentre:

"Approvo: da ora X viene sostituito da Y"

può costituire un delta decisionale, se proviene dall'authority corretta.

Il ciclo della memoria non deve trasformare intenzione in authority per interpretazione creativa.

---

## 6.5 Causal Impact Check: se cambia questo, cos'altro potrebbe cambiare?

Un delta materiale raramente vive isolato.

Una nuova decisione può modificare:

- requisiti;
- processi;

- documentazione;
- stato;
- roadmap;
- output;
- relazioni;
- indici;
- workflow.

Per questo **PROC-006** include un **Causal Impact Check**.

La domanda diventa:

**| Quali altri nodi dipendono materialmente da ciò che è cambiato?**

La risposta forma l'**Impact Set**.

L'Impact Set non è “tutto ciò che potrebbe essere vagamente collegato”.

È il perimetro minimo degli elementi che devono essere verificati o aggiornati affinché il nuovo stato sia coerente.

Esempio:

```
DECISIONE A cambia
      ↓
PROCESSO B dipende da A
DOCUMENTO C descrive B
INDICE D punta alla versione corrente
```

Se aggiorniamo soltanto A, il sistema può diventare incoerente.

La memoria persistente corretta non è quindi una collezione di write indipendenti.

È una rete che deve preservare coerenza attraverso le dipendenze materiali.

## 6.6 Consolidation: trasformare il delta in memoria organizzativa

La **Consolidation** è il passaggio nel quale il delta classificato e autorizzato viene scritto nella destinazione persistente appropriata.

La destinazione dipende dalla natura del delta.

Esempi:

| Tipo di delta       | Destinazione tipica                  |
|---------------------|--------------------------------------|
| decisione           | Decision Record / registro decisioni |
| stato corrente      | state source appropriata             |
| workflow checkpoint | runtime/workflows                    |
| requisito           | requirements/spec                    |
| evidence            | evidence / telemetry / result        |

|                  |                                      |
|------------------|--------------------------------------|
| relazione        | relationship ledger / typed relation |
| learning         | Method Experience Memory             |
| output approvato | output canonico/frozen               |
| superseded       | storico preservato con lineage       |

La regola fondamentale è:

**Persisti il significato organizzativo, non la forma conversazionale che lo ha prodotto.**

La frase:

*“Sì, va bene, facciamo Y invece di X”*

può diventare una struttura persistente che rappresenta:

- decisione Y;
- authority;
- data;
- decisione X superseded;
- impatti;
- provenance.

La conversazione è stata il luogo in cui il significato è nato.

La memoria persistente è il luogo in cui quel significato viene reso ricostruibile.

## 6.7 Perché non si copia la chat

Questa regola merita di essere ripetuta perché è centrale.

L'anti-pattern è:

```
INTERAZIONE
→ COPIA / RIASSUMI TUTTO
→ KB
```

Il pattern WCM è:

```
INTERAZIONE
→ COSA È CAMBIATO?
→ CHE TIPO DI DELTA È?
→ HA AUTHORITY?
→ COSA IMPATTA?
→ DOVE APPARTIENE?
→ CONSOLIDA
```

Perché?

Perché il valore della memoria organizzativa non è riprodurre fedelmente ogni pensiero.

È permettere a una futura sessione di ricostruire:

- cosa vale;
- perché;

- con quale status;
- chi lo ha autorizzato;
- cosa è stato superato;
- che cosa dipende da esso.

Una trascrizione completa può essere evidence storica utile.

Ma non è, da sola, la rappresentazione operativa del sistema.

---

## 6.8 Consistency Bundle: quando il consolidamento è davvero completo

Scrivere il Persistent Target corretto non basta.

Dopo una mutazione materiale, **PROC-006** richiede un **Consistency Bundle Check**.

Il bundle verifica almeno:

- source of truth del delta;
- eventuale workflow checkpoint;
- current-facing mirrors;
- indici;
- registri;
- decisioni collegate;
- relazioni;
- living ledger;
- documentation projections.

Il principio è:

**Consolidato  $\neq$  scritto. Consolidato = scritto + propagato dove necessario + verificato.**

Questo evita un failure molto comune:

```

FONTE CORRETTA
+
INDICE STALE
+
MIRROR STALE
+
CHECKPOINT STALE
=
MEMORIA INCOERENTE
```

Quando il bundle non è verde, il ciclo non deve fingere di essere completo.

Il sistema deve registrare drift e passare all'assurance appropriata.

---

## 6.9 Assurance: il ciclo di memoria non termina con la scrittura



PROC-006 consolida.

PROC-008 Knowledge Integrity Assurance Loop verifica che la memoria risultante sia ancora integra e osservabile.

Sono due funzioni diverse.

Possiamo rappresentarle così:

```
PROC-006
"Ho trasferito e propagato il delta?"

↓

PROC-008
"La memoria risultante è coerente e verificata?"
```

L'assurance può controllare deterministicamente:

- state consistency;
- decision propagation;
- relationship validity;
- ledger freshness;
- orphan control;
- index reachability;
- freshness dell'ultimo health check.

Se tutto è verde, il sistema può fermarsi.

Se emerge un'anomalia meccanica e allowlisted, può essere applicata una repair deterministica.

Se il problema è semantico o ambiguo, il sistema non deve inventare una soluzione.

Deve escalare.

---

## 6.10 Persistent → Working: come la memoria torna utile nel presente

Il secondo movimento della Dual Memory parte dalla direzione opposta.

Una nuova richiesta arriva.

La Working Memory corrente può possedere già parte del contesto.

Ma ciò che serve potrebbe trovarsi nella memoria persistente.

WCM deve allora recuperare **soltanto il contesto necessario**.

Il problema non è:

| *“Che cosa abbiamo in archivio?”*

È:

| **“Che cosa mi manca per svolgere correttamente questo task?”**

Questo cambio di domanda è fondamentale.

La Persistent Organizational Memory non viene “caricata”.

Viene **interrogata progressivamente**.

---

## 6.11 Context-aware bootstrap: partire da ciò che è già disponibile

**PROC-005 Agent-Ready Context Bootstrap** stabilisce che il sistema deve considerare prima il contesto pertinente già disponibile nella Working Memory.

Questo evita un altro anti-pattern:

```
NUOVA RICHIESTA
→ IGNORA TUTTO CIÒ CHE SAI GIÀ
→ RILEGGI TUTTO
```

Se goal, scope, vincoli e riferimenti sono già chiari, il sistema può usarli.

Ma non deve confondere comodità con authority.

Per elementi sensibili come:

- decisioni frozen;
- stato operativo;
- workflow checkpoint;
- governance;
- authority;
- processi e protocolli correnti;

può essere necessaria una verifica persistente.

Il bootstrap è quindi **context-aware**.

Non è né chat-first né repository-first in senso assoluto.

---

## 6.12 INDEX-FIRST: trovare la strada prima di leggere i documenti

Quando serve retrieval persistente, WCM applica **PROT-005 Index-First Progressive Retrieval**.

La sequenza è:

```
L0 – ENTRY POINT
    ↓
L1 – INDEX / MAP
    ↓
L2 – ACTIVE AUTHORITY / PROCEDURE
    ↓ se necessario
L3 – EVIDENCE / HISTORY / RAW
```

Il principio è:

**| Navigate first, retrieve progressively, stop when sufficient.**

L'indice non contiene necessariamente la risposta.

Contiene la **mappa verso la risposta**.

Questa distinzione consente alla memoria di crescere senza costringere ogni nuova sessione a ricostruire l'intera organizzazione.

---

## 6.13 Retrieval Gate: ogni lettura deve avere una ragione

Prima di aprire un nuovo documento, **PROT-005** impone quattro domande:

1. quale informazione manca?
2. questo file è probabilmente la fonte più autorevole?
3. l'informazione è già disponibile?
4. il task richiede davvero questo livello di dettaglio?

Se il sistema non sa rispondere, la lettura rischia di essere rumore.

Questo è uno dei punti più importanti per la scalabilità della memoria WCM.

Una memoria cresce bene non quando può essere letta tutta.

Cresce bene quando può essere **navigata selettivamente**.

---

## 6.14 Stop When Sufficient: sapere quando fermarsi

Il retrieval ha bisogno di una vera stop condition.

Altrimenti un sistema intelligente tende facilmente a continuare a cercare informazioni “per sicurezza”.

WCM considera il contesto sufficiente quando l'attore sa, nella misura richiesta dal task:

- chi è;
- qual è il goal;
- quale workflow eventualmente deve riprendere;
- quale source of truth è pertinente;
- quale authority possiede;
- quale scope è autorizzato;
- quali processi/protocolli si applicano;
- quale transizione viene dopo;
- quale **true stop**, cioè la condizione reale che autorizza la conclusione del workflow e non una semplice interruzione tecnica, deve raggiungere;
- quali gap restano aperti.

Quando queste risposte sono disponibili, nuove letture devono essere motivate.

Il principio è:

PIÙ CONTESTO  
≠

Dopo una certa soglia, nuovo contesto può aumentare rumore e contraddizioni.

---

## 6.15 Source Precedence: recuperare non significa credere a tutto nello stesso modo

Durante il retrieval possono emergere più fonti.

Non tutte hanno lo stesso peso.

Una fonte più recente non è automaticamente più autorevole.

Una nota può essere più nuova di una decisione frozen ma non sostituirla.

Una evidence può essere corretta senza essere authority.

Un concept può essere interessante senza essere baseline.

Per questo la memoria persistente deve essere letta secondo source precedence.

In forma semplificata:

```
GOVERNANCE / MANDATE
↓
CANON / ACTIVE BASELINE
↓
SPECIFIC CONTRACT
↓
VALIDATED PROCESS / PROTOCOL
↓
CURRENT STATE
↓
DECISION
↓
EVIDENCE
↓
OPEN CONCEPT
↓
RAW / HISTORICAL
```

Il retrieval quindi non risponde soltanto a:

**|** *“Dove trovo qualcosa?”*

Risponde anche a:

**|** **“Che autorità devo attribuire a ciò che ho trovato?”**

---

## 6.16 Contraddizioni apparenti tra Working e Persistent Memory

Una differenza tra le due memorie non è automaticamente un conflitto.

Esempio:

```
PERSISTENT
Decisione attiva = X

WORKING
"Potremmo valutare Y"
```

Non c'è contraddizione.

X resta decisione attiva.

Y è proposta.

Altro caso:

```
PERSISTENT
Decisione attiva = X

WORKING
Authority competente:
"Da ora adottiamo Y"
```

Qui nasce un nuovo delta materiale.

La memoria persistente è corretta rispetto al passato.

La Working Memory contiene un nuovo cambiamento che deve essere consolidato.

Il sistema non deve scegliere "chi ha ragione".

Deve classificare il nuovo contenuto e applicare il processo di modifica.

---

## 6.17 Contraddizioni reali tra fonti persistenti

Più delicato è il caso in cui due fonti persistenti apparentemente autorevoli dichiarano cose incompatibili.

Esempio:

```
Fonte A
Stato = APPROVED

Fonte B
Stato = DRAFT
```

Se entrambe sembrano current-facing e autorevoli, WCM non deve mediare a intuito.

**PROT-005** stabilisce:

**| No silent conflict resolution.**

Il sistema deve:

1. determinare la source precedence applicabile;
2. verificare status e provenance;
3. capire se una fonte è stale o superseded;
4. se il conflitto resta realmente autorevole, escalare.

Questo è un principio di sicurezza.

Una memoria organizzativa affidabile deve poter ammettere:

| “Non so quale delle due fonti sia valida.”

invece di inventare una sintesi.

## 6.18 Delta Retrieval: nei follow-up non rileggere tutto

Il ciclo Dual Memory è particolarmente efficiente quando il lavoro prosegue in più passaggi.

Dopo un bootstrap iniziale, una nuova interazione non dovrebbe comportare ogni volta la ricostruzione completa del contesto.

**PROT-005** introduce il principio:

### | Delta preferred.

Nei follow-up il sistema dovrebbe privilegiare il delta, salvo audit, verifica o conflitto che richiedano una rilettura più ampia. Dovrebbe quindi chiedere:

COSA È CAMBIATO  
DAL CONTESTO GIÀ SUFFICIENTE?

piuttosto che:

RILEGGI TUTTO

Questo riduce:

- token;
- latenza;
- rischio di confondere storico e corrente;
- letture ridondanti;
- costo operativo.

## 6.19 Il ciclo completo durante una sessione lunga

Vediamo ora come i due movimenti possono alternarsi più volte all'interno dello stesso lavoro.

1. Working Memory contiene il task corrente
2. Manca una decisione  
↓  
Retrieval persistente
3. La decisione entra nella Working Memory
4. Il lavoro produce una nuova evidence  
↓  
Delta Detection

- 5. Evidence classificata  
↓  
Consolidation
- 6. Nuova decisione necessaria  
↓  
Retrieval di authority / vincoli
- 7. Decisione autorizzata  
↓  
Consolidation + Impact Set
- 8. Assurance  
↓  
Working Memory continua

La Dual Memory non opera quindi soltanto “a fine sessione”.  
È un circuito che può attivarsi ogni volta che il lavoro lo richiede.  
La frequenza non è fissa.  
Dipende dalla materialità dei delta e dalla necessità di retrieval.

## 6.20 Fine sessione ≠ fine memoria

Una sessione può terminare mentre un workflow è ancora incompleto.  
Questo non deve spezzare il ciclo.  
Se il lavoro ha prodotto transizioni materiali, il checkpoint persistente deve rappresentare il punto raggiunto.  
Alla sessione successiva:

```
BOOTSTRAP
→ WORKFLOW CHECKPOINT
→ RESUME PRIORITY
→ RETRIEVAL MINIMO
→ CONTINUA
```

La continuità non deriva dalla speranza che il modello “ricordi”.  
Deriva dal fatto che la memoria persistente contiene il checkpoint necessario e che il bootstrap sa trovarlo.  
Questo principio trasforma:

```
FINE CHAT
```

da possibile perdita di continuità a semplice interruzione tecnica.

## 6.21 Knowledge Health e freshness: quando la memoria può essere considerata affidabile

Dopo un delta materiale, l'ultimo check di Knowledge Health precedente può non essere più sufficiente.

**PROT-013** stabilisce l'invariante:

```
LAST_KNOWLEDGE_CHECK  
<  
LAST_MATERIAL_DELTA  
  
⇒ HEALTHY VIETATO
```

Questo significa che una memoria può essere sostanzialmente corretta ma avere stato **STALE** finché non viene verificata.

La distinzione è importante.

WCM non vuole rappresentare la fiducia come impressione.

Vuole collegarla a evidence di verifica sufficientemente fresca.

Gli stati principali sono:

- HEALTHY;
- DEGRADED;
- STALE;
- CRITICAL;
- UNKNOWN.

Un badge verde non è un'opinione.

Deve essere supportato da un check corrente rispetto all'ultimo delta materiale. **HEALTHY** significa quindi che gli invarianti dichiarati risultano soddisfatti e sufficientemente freschi rispetto all'ultimo delta noto; non significa certezza assoluta che l'intera organizzazione non contenga alcun errore possibile.

---

## 6.22 Cosa succede se l'assurance trova un problema

Se l'assurance trova drift, il sistema distingue due categorie.

### Problema meccanico deterministico

Esempio astratto:

- un fingerprint atteso non è aggiornato;
- la fonte autorevole è univoca;
- il resto dei mirror è già coerente;
- esiste una repair class allowlisted.

Il sistema può correggere automaticamente e rieseguire il check.

### Problema semantico

Esempio:



- due decisioni autorevoli sembrano incompatibili;
- una relazione richiede interpretazione;
- non è chiaro quale versione debba essere corrente;
- correggere significherebbe cambiare canone o strategia.

In questo caso:

```
NO WRITE  
→ WISE / GATE / AUTHORITY
```

La memoria non viene “aggiustata” finché passa il test.

Viene corretta soltanto quando esiste una base determinabile o un'authority appropriata.

---

## 6.23 Il ciclo non è una burocrazia continua

A questo punto il sistema potrebbe sembrare molto pesante.

In realtà la baseline WCM contiene una regola opposta.

**PROC-006** dice esplicitamente:

**| Non applicare il consolidamento come rituale dopo ogni messaggio.**

Un breve ragionamento locale non richiede:

- Impact Set;
- Decision Record;
- assurance;
- aggiornamento indice;
- nuovo ledger.

Il ciclo completo si attiva quando esiste un delta materiale o una necessità reale di retrieval.

La disciplina serve a evitare errori organizzativi.

Non a trasformare ogni interazione in una procedura amministrativa.

---

## 6.24 Due errori speculari

L'intera Dual Memory può essere compresa attraverso due failure opposti.

### Errore A — Tutto resta vivo

```
Working Memory  
→ nessun consolidamento  
→ sessione termina  
→ perdita di continuità
```

Il sistema pensa bene nel presente ma dimentica il futuro.

## Errore B — Tutto diventa archivio

Working Memory  
→ persistenza indiscriminata  
→ repository cresce  
→ retrieval onnivoro  
→ rumore

Il sistema ricorda molto ma fatica a sapere cosa conta.

WCM prova a stare nel mezzo:

RICCHEZZA COGNITIVA  
+  
PERSISTENZA SELETTIVA  
+  
RETRIEVAL SELETTIVO

## 6.25 La vera continuità: stateful organization + context-aware cognition

Possiamo ora comprendere pienamente una delle formule centrali della Dual Memory:

### **stateful organization + context-aware cognition**

**Stateful organization** significa che l'organizzazione possiede stato persistente, decisioni, workflow, authority, evidence e storia che non dipendono dalla singola sessione.

**Context-aware cognition** significa che il sistema non ignora il contesto vivo disponibile e non rilegge meccanicamente tutto da zero.

Le due proprietà cooperano:

PERSISTENZA SENZA COGNIZIONE CONTESTUALE  
= rigida / costosa / rumorosa

COGNIZIONE SENZA PERSISTENZA  
= ricca / fragile / volatile

WCM  
= cooperazione delle due

Questa è la logica fondamentale della Dual Memory.

## 6.26 Una sessione futura deve poter riprendere senza la chat precedente

Possiamo formulare una prova pratica molto semplice.

Supponiamo che la sessione corrente termini adesso.

Una sessione futura dovrebbe poter ricostruire:

- stato corrente;

- decisioni attive;
- workflow incompleti;
- next transition;
- authority;
- vincoli;
- output frozen;
- evidence necessaria;
- processi/protocolli applicabili;

senza dipendere dalla trascrizione integrale della conversazione precedente.

Allo stesso tempo, non dovrebbe essere costretta a leggere tutta la memoria persistente.

Dovrebbe poter usare:

```
ENTRY POINT
→ INDEX
→ FONTI NECESSARIE
→ STOP WHEN SUFFICIENT
```

Se entrambe queste condizioni sono soddisfatte, il ciclo Dual Memory sta funzionando.

## 6.27 Quando la memoria organizzativa può essere considerata coerente

**PROC-006** definisce criteri molto concreti.

Il consolidamento è completo quando:

- ciò che deve sopravvivere è persistito;
- authority e status sono chiari;
- il workflow checkpoint è corrente quando applicabile;
- lineage e precedenti sono ricostruibili;
- gli impatti materiali dichiarati sono verificati;
- le sinapsi necessarie sono aggiornate;
- i mirror pertinenti non sono in conflitto;
- una futura sessione può ricostruire stato e next transition;
- l'assurance è corrente oppure la memoria dichiara esplicitamente uno stato non-green.

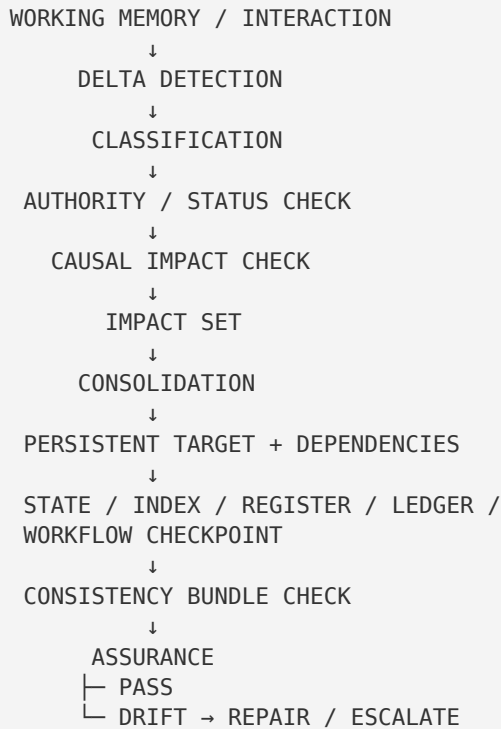
Questa è una definizione più forte di:

**|** *"i file sembrano a posto".*

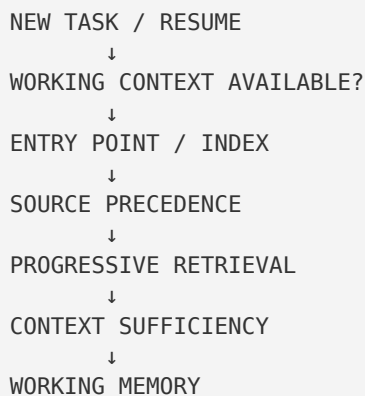
La coerenza è una proprietà del sistema di relazioni, non soltanto dei singoli documenti.

## 6.28 Il Memory Consolidation Loop in forma completa

Possiamo ora leggere il processo come un unico flusso.



E sul percorso opposto:



Questi due flussi formano il circuito completo.

## 6.29 Perché questo ciclo è importante per il WCM

La Dual Memory risolve contemporaneamente quattro problemi.

### Continuità

Il lavoro può attraversare sessioni senza dipendere dalla memoria volatile.

## Efficienza

La memoria persistente non viene caricata integralmente.

## Governance

Decisioni, authority, stato ed evidence non vengono confusi.

## Apprendimento

L'esperienza può essere consolidata e riutilizzata senza diventare automaticamente baseline.

Queste quattro proprietà rendono il ciclo molto più di un meccanismo di “memoria AI”.

È una forma di **continuità organizzativa governata**.

---

## 6.30 Dove siamo arrivati

Chiudiamo la Parte II con dodici idee essenziali.

1. La Dual Memory è un ciclo, non due contenitori.
2. Working → Persistent inizia con la Delta Detection.
3. Il delta viene classificato prima di essere persistito.
4. Authority e status devono essere verificati.
5. I cambi materiali richiedono un Causal Impact Check proporzionato.
6. Il consolidamento persiste il significato organizzativo, non la chat.
7. Il consolidamento non è completo senza Consistency Bundle.
8. L'assurance verifica freshness, relazioni e coerenza risultante.
9. Persistent → Working avviene tramite retrieval progressivo.
10. INDEX-FIRST evita full reload e rumore.
11. Il retrieval termina quando il contesto è sufficiente.
12. La continuità WCM nasce da **stateful organization + context-aware cognition**.

Con questo capitolo si chiude il primo grande blocco concettuale del libro.

Ora sappiamo:

- che cos'è la Working Memory;
- che cos'è la Persistent Organizational Memory;
- come un delta passa dall'una all'altra;
- come la memoria persistente torna nel contesto vivo;
- come WCM prova a mantenere le due coerenti.

Nel prossimo blocco faremo un ulteriore passo.

Scopriremo perché la memoria WCM non viene concepita soltanto come una gerarchia di file, ma come una **rete di nodi e relazioni**.

---

# Source Map — Draft 06

Fonti canoniche principali usate per questa stesura:

- [wcm/kb/decisions/DEC-004\\_DUAL\\_MEMORY\\_CAUSAL\\_DECISION\\_BASELINE.md](#) — Dual Memory FROZEN come principio architetturale e Memory Consolidation Loop selettivo;
- [wcm/kb/concepts/CONCEPT-008\\_DUAL\\_MEMORY\\_COGNITIVE\\_CONTINUITY.md](#) — ciclo Working ↔ Persistent, delta, classification, consolidation e context-aware retrieval;
- [wcm/process-book/processes/PROC-005\\_AGENT\\_READY\\_CONTEXT\\_BOOTSTRAP.md](#) — Working Memory pertinente, Resume Priority, Context Sufficiency e retrieval minimo;
- [wcm/process-book/processes/PROC-006\\_MEMORY\\_CONSOLIDATION\\_LOOP.md](#) — Delta Detection, Classification, Authority/Status Check, Impact Set, Consolidation e Consistency Bundle;
- [wcm/process-book/protocols/PROT-005\\_INDEX\\_FIRST\\_PROGRESSIVE\\_RETRIEVAL.md](#) — L0/L1/L2/L3, retrieval gate, source precedence, stop when sufficient e delta preferred;
- [wcm/process-book/processes/PROC-008\\_KNOWLEDGE\\_INTEGRITY\\_ASSURANCE\\_LOOP.md](#) — assurance post-delta, deterministic-first e controlled auto-repair;
- [wcm/kb/concepts/CONCEPT-011\\_KNOWLEDGE\\_SYNAPSE\\_ASSURANCE.md](#) — Knowledge Health, freshness invariant e relazioni operative;
- [wcm/process-book/protocols/PROT-013\\_KNOWLEDGE\\_SYNAPSE\\_HEALTH\\_STANDARD.md](#) — stati HEALTHY/STALE/DEGRADED/CRITICAL e health freshness.

## Figura collegata

- [FIG-001B\\_DUAL\\_MEMORY\\_ARCHITECTURE.svg](#) — APPROVED / riusata come figura architetturale principale del capitolo.

## Note per la Technical Review

Verificare in particolare:

- che consolidation non venga descritta come salvataggio integrale della chat;
- che Delta Detection e Classification restino coerenti con PROC-006;
- che authority/status precedano la persistenza materiale;
- che Impact Set e Consistency Bundle non vengano presentati come obbligo per ogni micro-interazione;
- che PROC-006 e PROC-008 restino distinti;
- che context-aware bootstrap non diventi chat-first assoluto;
- che INDEX-FIRST e source precedence restino aderenti a PROT-005;
- che Knowledge Health **HEALTHY** richieda freshness rispetto all'ultimo delta materiale;
- che nessun esempio o riferimento project-specific entri nel capitolo.